claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_e1b943dd-dac\agent-a21c7266377a4ba9c.jsonl`
- SHA-256: `a560f221e29f6db0d72fac9dbeac874b73a70a303a1648f74d89d2231888af53`
- Source modified: `2026-07-06T19:19:28+00:00`
- Imported at: `2026-07-08T16:00:11+00:00`
- project: `wf_e1b943dd-dac`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T19:18:19.571Z)

```
Reviewing GitHub PR #40 of the repo at C:/Users/avidu/Projects/Telegram ? a DRAFT, DOCS-ONLY PR
(branch docs/track-f-policy-pipeline).
It adds ONE file: docs/adr/0012-policy-pipeline.md (ADR 0012, status Proposed) ? the Track F
design doc for a
four-tier "policy pipeline" (Tier 1 metadata gates / Tier 2 topic preferences / Tier 3 content
safety / Tier 4
deep inspection), a PolicyStage protocol + StageResult explanation trace, and 7 phased follow-up
issues F1-F7.
No code changes. Read the ADR:
  git -C C:/Users/avidu/Projects/Telegram show docs/track-f-policy-
pipeline:docs/adr/0012-policy-pipeline.md

This is a DESIGN review, not a code review ? there is no gate to run. Judge the ADR as a design
contract:
soundness, completeness, internal consistency, accuracy of its claims, and consistency with the
codebase and the
other ADRs (docs/adr/0001..0011 exist on main; read any you need via 'git -C
C:/Users/avidu/Projects/Telegram show main:docs/adr/<file>').
Relevant current code (on main): src/tg_video_filter/models.py (FilterConfig, ContentItem,
Source, Policy),
src/tg_video_filter/db.py (load_filter_config/save_filter_config shims, sources.watched gate
should_ingest_source,
delivery_attempts/record_delivery), src/tg_video_filter/telethon_client.py (the live handler:
watched gate -> dedup
-> FilterEngine.evaluate -> insert -> deliver). Track E (sources.watched opt-in gate) and Track
C (delivery_attempts)
just merged.

Ground rules:
- READ actual files, cite ADR section / file:line. You MAY run
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe to check claims (e.g. does
FilterConfig
  ignore/drop unknown JSON keys? does ContentItem carry caption text? is content_item.text
populated?).
- This is a DRAFT asking for design feedback ? frame findings as design gaps/risks/inaccuracies,
not merge-blockers.
  Severity: high (a claim that is false, or a design gap that would break an F-phase / cause
data loss), medium
  (unspecified interaction, inconsistency, missing dependency), low (wording, invalid example,
minor).
- Report only issues in THIS ADR. No praise. Be concrete: what's wrong, why it matters for
implementation.
```

ADVERSARIALLY VERIFY this design finding ? try to REFUTE it. Read the cited ADR section + any referenced ADR/code,
run C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe checks if useful. CONFIRMED if it genuinely holds against THIS ADR (a real gap/inaccuracy/inconsistency).
REFUTED if the ADR actually addresses it / the claim is wrong / it's out of scope for a design doc. PLAUSIBLE if
unsettled. Corrected severity (remember: draft design doc, so calibrate ? false claim/data-loss = high, gap = medium).
FINDING (design): StageResult recorded via delivery_attempts.last_error 'for failed items' is a category error ? rejected items short-circuit before any delivery_attempts row exists
severity high | section Decision ? Policy evaluation flow (records results via delivery_attempts.last_error 'for failed items')
summary: delivery_attempts rows are keyed (content_item_id, destination_id) and created by record_delivery at delivery time. A policy-rejected item short-circuits BEFORE delivery, so there is no destination and no delivery_attempts row to write last_error into.
why_it_matters: The ADR proposes storing the failed-stage explanation in delivery_attempts.last_error. But a rejected item is never delivered ? record_delivery(content_item_id, destination_id, success=False, error=...) requires a destination_id and is called only from the delivery path. For a Tier-1/2/3 rejection there is no (item,destination) pair, so this storage target doesn't exist for exactly the items whose reason you want to record. It also conflates 'delivery transport failed' with 'policy rejected' in one column. The only viable store for rejections is the deferred policy_decisions table ? which means F6 is a hard dependency of any persisted explainability, and F7's /why has nothing to read until F6.
evidence: db.py:1339 record_delivery signature requires content_item_id AND destination_id; last_error lives on delivery_attempts UNIQUE(content_item_id,destination_id). telethon_client.py delivers only when result.matched; a rejected item never reaches deliver()/record_delivery. ADR flow itself returns False and short-circuits before delivery.
rec: Drop the delivery_attempts.last_error path for policy rejections. State that the explanation trace for rejected items has NO persistence in F1?F5 (in-memory/log only) and that persisted explainability requires the policy_decisions table (F6). Reflect this in the phasing dependencies.

2. assistant (2026-07-06T19:18:23.287Z)

I'll verify this finding by examining the ADR and the relevant code claims.

3. user (2026-07-06T19:18:25.385Z)

ADR 0012: Policy pipeline ? explainable, tiered, user-customizable content gates

- **Status**: Proposed
- **Date**: 2026-07-06
- **Decision owner**: project owner
- **Tracking issue**: #31 (Track F of the provider-neutral umbrella, #25)
- **Depends on**: ADR 0011 (Track B ? domain schema, `policies` table)

Context

The system today evaluates every matched video against a single `FilterConfig` (per account, via `policies.config_json`). The filter is **flat**: all rules run unconditionally, the output is a boolean (`matched`), and there is no way to express topic preferences, security policy, or why a specific video passed or failed.

Track F defines a **policy pipeline** ? an ordered sequence of stages that run before delivery ? so the system can express richer decisions, explain them, and defer expensive work until after cheap rejections.

What we have today (post-Track-E)

Concept	Current state

`FilterConfig` / `Policy`	One JSON blob per account. Flat duration/resolution/size/exclude rules. AND semantics.
`FilterEngine`	Evaluates all rules; returns `FilterResult(passed, reasons)`.
`policies` table	One row per account (is_default=1). `config_json` is the `FilterConfig` blob.
Security / topic / deep-inspect	Nothing. Codec/keyword/MIME filters and ffprobe were deferred to v2 in the original plan.

What we want

A **policy** should be a pipeline of ordered **stages**, each with a well-defined cost, inputs, and explainable output. The pipeline runs **after** dedup and **before** delivery. Users should be able to express topic preferences ("I like AI and philosophy videos"), security rules ("reject links from sketchy domains"), and view **why** a specific item was accepted or rejected.

Decision

Stage model ? four ordered tiers

The pipeline is split into four tiers, ordered by cost (cheapest first). Later stages only run if all earlier stages pass:

```
????????????????????????????????????????????????????????????????
?                    Policy Pipeline                           ?
?                                                              ?
?  Tier 1 ? Metadata gates  (always runs, zero I/O)           ?
?     duration, resolution, size, codec, mime, GIF/round       ?
?     ? extends today's FilterEngine                           ?
?                                                               ?
?  Tier 2 ? Topic preferences  (opt-in, metadata-only)         ?
?     source allowlist, keyword match, channel topic tags       ?
?     ? user-expressed interests filter the stream              ?
?                                                               ?
?  Tier 3 ? Content safety  (opt-in, metadata + light I/O)     ?
?     URL/domain reputation, file metadata sanity,              ?
?     caption text scanning                                     ?
?     ? cheap checks; no download                               ?
?                                                               ?
?  Tier 4 ? Deep inspection  (opt-in, heavy I/O)               ?
?     ffprobe, perceptual hash, downloaded content scan         ?
?     ? gated behind explicit user opt-in + budget              ?
????????????????????????????????????????????????????????????????
```

Stage interface ? `PolicyStage`

Each stage is a pure function (or async function for I/O stages) that takes a `PolicyContext` and returns a `StageResult`:

```python
@dataclass
class PolicyContext:
    """Immutable snapshot of what the stage can inspect."""
    content_item: ContentItem        # from ADR 0011
    source: Source                   # the source it came from
    policy: Policy                   # the owning policy config

@dataclass
class StageResult:
    passed: bool
    reason: str                      # human-readable explanation
```

```
    stage_name: str                  # e.g. "duration-gate", "url-reputation"
    metadata: dict[str, object] = field(default_factory=dict)  # for explainability UI

class PolicyStage(Protocol):
    """A single gate in the pipeline. Stateless; may be async for I/O stages."""
    async def evaluate(self, ctx: PolicyContext) -> StageResult: ...
```

The pipeline evaluates stages in order and **short-circuits** on the first failure (don't waste I/O on an already-rejected item). The accumulated `StageResult` list is the **explanation trace** ? every decision is auditable.

Topic preferences ? Tier 2

Topic preferences are stored as a JSON array in `policies.config_json` under a `topics` key:

```json
{
  "min_duration_s": 5,
  "topics": {
    "include": ["AI", "Philosophy", "Soccer"],
    "exclude": ["Politics", "Crypto"],
    "mode": "any"  // "any" = match any include topic; "all" = match all
  }
}
```

Topic matching is **metadata-only** in v1: source title substring match, message caption keyword match, and (future) channel topic tags from Telegram's channel metadata. No ML classification in the design ? that's a separate issue.

Security stages ? Tier 3

Security stages are **opt-in** and configured per-policy. The design defines three initial stages (implementation deferred to separate issues):

1. **Source allowlist**: reject items from sources not in the `allowlist` (an explicit list of trusted provider_source_ids).
2. **URL/domain reputation**: check links in captions against a configurable blocklist. Metadata-only ? no page fetch.
3. **Duplicate suppression**: enhanced dedup that compares by `content_fingerprint` across accounts (already handled by `content_items.dedup_key`; this stage adds cross-policy duplicate detection within an account's delivery streams).

Each security stage records its decision in `StageResult.metadata` for explainability (e.g. `{"blocked_domain": "sketchy-site.com"}`).

Deep inspection ? Tier 4

Deep inspection (ffprobe, perceptual hash, downloaded content scan) is **gated behind explicit opt-in** because it violates ADR 0003's no-download principle. The opt-in is per-policy (`deep_inspect: true` in `config_json`) and the stage has a **budget** (max bytes per item, max concurrent downloads) enforced by the policy engine.

This tier is explicitly deferred to a follow-up issue ? the design only **reserves the slot** in the pipeline model so it doesn't require a schema migration when implemented.

Policy evaluation flow (post-dedup, pre-delivery)

```python
async def evaluate_pipeline(
    ctx: PolicyContext,
    stages: list[PolicyStage],
) -> tuple[bool, list[StageResult]]:
    results: list[StageResult] = []
    for stage in stages:
        result = await stage.evaluate(ctx)
        results.append(result)
        if not result.passed:
            return False, results  # short-circuit
    return True, results
```

The caller (telethon_client or future Bot API handler) records the
results via `delivery_attempts.last_error` (for failed items) or a new
`policy_decisions` table (for explainability ? deferred to implementation).

Schema ? minimal changes

No schema migration is required for this design. The `policies` table
already holds a `config_json` JSON blob. The pipeline is a code concept:

- `policies.config_json` gains optional `topics`, `allowlist`,
  `security_checks`, `deep_inspect` keys ? all backwards-compatible with
  the existing `FilterConfig` Pydantic model (extra fields are ignored).
- Future `policy_decisions` table (deferred) would store the explanation
  trace for debugging: `(content_item_id, policy_id, stage_name, passed,
  reason, metadata_json, evaluated_at)`.

Implementation phasing (future issues)

Phase	Scope	Dependencies
F1 ? Policy engine prototype	`PolicyStage` protocol, `evaluate_pipeline()`, wire into live-message handler	None
F2 ? Topic preferences	`topics` config parsing, source title/caption keyword matching	F1
F3 ? Source allowlist	`allowlist` config, skip non-allowlisted sources	F1
F4 ? URL/domain reputation	Domain blocklist, caption link extraction	F1
F5 ? Deep inspection	ffprobe/perceptual hash, download budget enforcement	F1, ADR 0003 waiver
F6 ? Policy decisions table	Audit trail storage, explainability UI	F1
F7 ? Explainability endpoint	`/why <content_item_id>` command showing the explanation trace	F6

Each phase is a separate, mergeable PR ? none are bundled here. This ADR
only defines the **design contract** the phases code against.

Consequences

Positive
- Unifies topic, security, and quality gates into a single explainable
  framework ? no ad-hoc if-chains scattered across handlers.
- Cheap gates always run first; expensive enrichment is only reached for
  items that pass the cheap gates.
- `StageResult` list provides an audit trail (explainability) without
  additional DB lookups at decision time.
- Backwards-compatible: `policies.config_json` already exists; adding
  optional keys requires no migration.
- Each implementation phase is independently shippable behind feature
  flags.

Negative

- Four tiers add conceptual complexity vs. today's flat filter.
- `PolicyStage` protocol requires all stages to conform to the same
  interface, which may be awkward for deeply different checks
  (metadata-only vs. I/O-heavy). Mitigated by the `metadata` dict
  catch-all.
- Topic matching is string-based substring matching in Phase 1 ? not
  semantic. A future ML stage would replace it, but it's good enough for
  a v1.
- Deep inspection violates ADR 0003 and must be explicitly opt-in with a
  budget to avoid runaway costs.

Alternatives considered

- **Do nothing; keep flat FilterConfig.** Rejected ? the umbrella exists
  to dissolve the conflation of quality, topic, and security into a
  single boolean. A pipeline is the natural generalization.
- **Inline each stage in the handler** (no protocol). Rejected ? makes
  it impossible to test stages independently, reorder them, or add new
  ones without touching the hot path.
- **Use a rules engine / DSL.** Rejected ? over-engineered for <100
  users. A Python list of callables is simpler, testable, and
  introspectable.
- **Bundle all phases into a single PR.** Rejected ? scope discipline.
  Each phase is independently shippable; this ADR defines the shared
  interface they code against.

Open questions for the decision owner

1. **`StageResult.metadata` shape** ? dict or typed per-stage? Recommend:
   dict for Phase 1 (flexible), typed in Phase 2 (once the shape
   stabilizes).
2. **Policy decisions table** ? store every decision, or only failures?
   Recommend: every decision (explainability), with a TTL-based cleanup
   for old traces.
3. **Topic matching** ? substring only, or regex/glob? Recommend:
   substring (case-insensitive) for v1; natural-language matching in a
   follow-up.
4. **Deep inspection budget** ? per-item byte limit or per-account
   bandwidth cap? Recommend: per-item limit (10 MB default) with a
   per-account monthly cap enforced at the app level.

References

- ADR 0003 ? metadata-first probing (the no-download principle that Tier
  4 must opt in to violate).
- ADR 0009 ? selective channel scanning (the `topics.allowlist` concept
  generalizes `sources.watched`).
- ADR 0011 ? provider-neutral domain schema (the `policies` table this
  design extends, `ContentItem` as the `PolicyContext` input).
- Issue #31 ? Track F (this ADR's tracking issue).
- Issue #25 ? umbrella; this is the final track.

4. assistant (2026-07-06T19:18:27.399Z)

Now let me verify the key claims about the code. Let me check the `record_delivery` signature
and `delivery_attempts` schema.

5. user (2026-07-06T19:18:29.784Z)

6:``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
40:    telegram_id INTEGER UNIQUE,
61:    UNIQUE (user_id, channel_id)
78:    UNIQUE (user_id, channel_id, message_id)

```
88:-- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
102:    UNIQUE (account_id, provider, provider_source_id)
124:    UNIQUE (account_id, source_id, provider_message_id),
125:    UNIQUE (account_id, dedup_key)
136:    UNIQUE (account_id, name)
140:CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default
152:    UNIQUE (account_id, provider, target)
164:    UNIQUE (account_id, source_id, destination_id)
167:CREATE TABLE IF NOT EXISTS delivery_attempts (
174:    last_error      TEXT,
178:    UNIQUE (content_item_id, destination_id)
347:        "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
384:            telegram_id  INTEGER UNIQUE,
443:                UNIQUE (user_id, channel_id)
465:                UNIQUE (user_id, channel_id, message_id)
493:        "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
509:    ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
619:    #    this row. UNIQUE(account_id, name) dedups idempotent re-runs.
632:    #    UNIQUE constraint only applies once rows are populated). On fresh DBs
635:        "CREATE UNIQUE INDEX IF NOT EXISTS idx_policies_account_default "
674:    Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
737:    # 3. Backfill delivery_attempts from previously-forwarded videos.
740:        INSERT OR IGNORE INTO delivery_attempts
926:        last_error=row["last_error"],
1173:      - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
1174:      - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
1339:async def record_delivery(
1356:        """INSERT INTO delivery_attempts
1358:              last_error, created_at, delivered_at)
1363:             attempt_count=delivery_attempts.attempt_count + 1,
1364:             last_error=excluded.last_error,
1366:                                ELSE delivery_attempts.delivered_at END""",
1386:        "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id = ?",
1652:    Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1795:    "record_delivery",
```

6. user (2026-07-06T19:18:34.048Z)

```
CREATE TABLE IF NOT EXISTS delivery_attempts (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    content_item_id INTEGER NOT NULL REFERENCES content_items(id),
    destination_id  INTEGER NOT NULL REFERENCES destinations(id),
    route_id        INTEGER REFERENCES routing_rules(id),
    status          TEXT NOT NULL DEFAULT 'pending',
    attempt_count   INTEGER NOT NULL DEFAULT 0,
    last_error      TEXT,
    created_at      TEXT NOT NULL,
    delivered_at    TEXT,
    next_retry_at   TEXT,
    UNIQUE (content_item_id, destination_id)
);
"""
async def record_delivery(
    conn: aiosqlite.Connection,
    *,
    content_item_id: int,
    destination_id: int,
    route_id: int | None = None,
    success: bool,
    error: str | None = None,
) -> None:
    """Record a delivery outcome for one (content_item, destination) pair.

    This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
    sets status=delivered; on failure, increments attempt_count and records the
```

```
    error. Idempotent: repeated calls update the same row.
    """
    status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
    await conn.execute(
        """INSERT INTO delivery_attempts
             (content_item_id, destination_id, route_id, status, attempt_count,
              last_error, created_at, delivered_at)
           VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
                   CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
           ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
             status=excluded.status,
             attempt_count=delivery_attempts.attempt_count + 1,
             last_error=excluded.last_error,
             delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
                               ELSE delivery_attempts.delivered_at END""",
        (
            content_item_id,
            destination_id,
            route_id,
            status.value,
            error,
            success,
            success,
        ),
    )
    await conn.commit()


async def get_delivery_attempt(
    conn: aiosqlite.Connection, content_item_id: int, destination_id: int
) -> DeliveryAttempt | None:
    """Read a delivery attempt row by its natural key (idempotency boundary)."""
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id = ?",
        (content_item_id, destination_id),
    ) as cur:
        row = await cur.fetchone()
    return _row_to_delivery_attempt(row) if row else None
```

7. assistant (2026-07-06T19:18:36.939Z)

Confirmed: `record_delivery` requires both `content_item_id` AND `destination_id` (NOT NULL, part of the UNIQUE key). Now let me check the telethon_client flow to confirm rejected items short-circuit before delivery.

8. user (2026-07-06T19:18:39.312Z)

```
7:      -> dedup via db.get_video()
8:      -> FilterEngine.evaluate(meta)
9:      -> db.insert_video(...) + delivery.forward_to_target(..., target=cfg.delivery_target)
12:``Message`` and returning a ``VideoMeta | None``. This isolates the only piece
36:from tg_video_filter.delivery import build_link_text, deliver
47:    """Extract ``VideoMeta`` from a Telethon ``Message``, or ``None`` if not a video.
71:    if not isinstance(media, Document):
72:        return None
76:    if not has_video_attr:
77:        return None
102:    return meta
128:    return client
152:    delivery_mode: DeliveryMode = DeliveryMode.FORWARD,
153:    delivery_target: str = "me",
158:      1. extracts VideoMeta + document_id (skips if not a video),
159:      2. dedups via two-tier content dedup (document.id, then metadata fingerprint),
160:         plus the existing (channel, message) instance dedup,
```

```
163:      4. inserts a VideoRecord row,
164:      5. delivers a match to ``delivery_target`` (see ADR 0005).
170:            return  # Telethon edge case: event has no message (shouldn't happen)
174:            return
178:            return  # not a video
189:        if not await db.should_ingest_source(conn, user.id, channel_id):
190:            return  # skip unwatched / unknown source
192:        # --- three-tier dedup (cheapest first) ---
197:            return
204:                    "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
213:                return
232:                    "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
244:                return
257:        result = engine_holder[ENGINE_KEY].evaluate(meta)
268:            matched=result.matched,
271:            inserted = await db.insert_video(conn, record)
273:            # Lost a dedup race: a concurrent delivery of the same file
274:            # (same document_id or same message instance) won the insert.
279:                "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
285:            return
287:            "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
291:            result.matched,
295:        if result.matched:
298:            if delivery_mode != DeliveryMode.FORWARD:
307:            await deliver(
311:                inserted.id,
312:                delivery_mode,
314:                target=delivery_target,
317:    return on_new_message
327:    This is the most reliable content-based dedup key: Telegram reuses the
334:        return getattr(media, "id", None)
335:    return None
342:    collapse them to a single int for dedup/storage. The exact subclass isn't
349:            return int(val)
351:    return -abs(hash(repr(peer)) % (2**31))
357:    return getattr(chat, "title", None) if chat is not None else None
361:    """Structural type annotation for the handler returned by make_*_handler."""
```

9. assistant (2026-07-06T19:18:43.286Z)

Confirmed. Line 295: `if result.matched:` ? delivery (and thus any `record_delivery` in the delivery path) happens ONLY when matched. A rejected item never reaches delivery. Let me verify `deliver` itself calls `record_delivery` and requires a destination.

10. user (2026-07-06T19:18:45.468Z)

```
131:    destination_id: int | None = None,
139:    If ``destination_id`` is provided AND ``write_delivery_attempt`` is True,
140:    a per-destination ``DeliveryAttempt`` is written (ADR 0011 Track C).
146:    if write_delivery_attempt and destination_id is not None:
147:        await db.record_delivery(
150:            destination_id=destination_id,
170:    destination_id: int | None = None,
178:    if write_delivery_attempt and destination_id is not None:
179:        await db.record_delivery(
182:            destination_id=destination_id,
195:async def deliver(
204:    destination_id: int | None = None,
215:    ``destination_id`` is the FK to ``destinations.id`` (ADR 0011 Track C).
216:    When provided, a per-destination ``DeliveryAttempt`` is written.
220:            client, link_text, conn, video_id, target, destination_id=destination_id
223:        # Both legs write to the same (content_item_id, destination_id) row.
224:        # Defer DB writes so we record ONE combined outcome per destination
239:            destination_id=destination_id,
248:            destination_id=destination_id,
```

```
252:        if destination_id is not None:
258:            await db.record_delivery(
261:                destination_id=destination_id,
279:        client, message, conn, video_id, target, destination_id=destination_id
```

11. assistant (2026-07-06T19:18:58.959Z)

This fully confirms the finding. `record_delivery` is only ever called from within the `deliver` path, always guarded by `destination_id is not None`, and only reached when `result.matched`. There is no code path that writes `last_error` for a policy-rejected item.

Now let me confirm the claim that the `policy_decisions` table is the only viable store and check the phasing dependency implication (F6 ? F7).

The ADR itself states (Policy evaluation flow): "records the results via `delivery_attempts.last_error` (for failed items) or a new `policy_decisions` table (for explainability ? deferred to implementation)."

Let me check the F-phasing table ? F7 (`/why`) depends on F6, but F6 is the `policy_decisions` table. The ADR does list F7 ? F6. Let me verify the finding's claim about phasing consistency.

The finding's key claims:
1. delivery_attempts is keyed (content_item_id, destination_id) ? **CONFIRMED** (db.py:178).
2. record_delivery requires destination_id ? **CONFIRMED** (db.py:1343-1345, NOT NULL, called only with `destination_id is not None`).
3. Rejected item short-circuits before delivery ? **CONFIRMED** (telethon_client.py:295 `if result.matched:`; evaluate_pipeline returns False and short-circuits).
4. So `delivery_attempts.last_error` cannot hold a rejection reason ? **CONFIRMED**.

Now let me consider possible refutations. Could a rejected item ever have a delivery_attempts row from a *prior* delivery? No ? dedup prevents re-processing, and a first-time rejection has no prior delivery. Could the ADR mean "delivery transport failed" items only (not policy rejections)? Let me re-read the exact ADR sentence.

The ADR sentence: "The caller ... records the **results** via `delivery_attempts.last_error` (for failed items) or a new `policy_decisions` table". The "results" are the `evaluate_pipeline` StageResults, and `evaluate_pipeline` returns `False` for policy-rejected items. "failed items" in context means policy-failed items (the pipeline returns `False, results`). So the ADR does propose storing policy-rejection reasons in `delivery_attempts.last_error`. That is the category error.

Let me double check whether "failed items" could plausibly be read as "delivery-failed items" ? a charitable reading that would refute.

12. user (2026-07-06T19:19:01.364Z)

```
15:passed or failed.
19:them, and defer expensive work until after cheap rejections.
36:("reject links from sketchy domains"), and view **why** a specific item was
37:accepted or rejected.
94:The pipeline evaluates stages in order and **short-circuits** on the first
95:failure (don't waste I/O on an already-rejected item). The accumulated
125:1. **Source allowlist**: reject items from sources not in the `allowlist`
161:            return False, results  # short-circuit
166:results via `delivery_attempts.last_error` (for failed items) or a new
167:`policy_decisions` table (for explainability ? deferred to implementation).
177:- Future `policy_decisions` table (deferred) would store the explanation
242:2. **Policy decisions table** ? store every decision, or only failures?
```

13. assistant (2026-07-06T19:19:12.694Z)

The ADR uses "failed"/"failure" throughout to mean policy-pipeline failure (line 15 "passed or failed", line 95 "short-circuits on the first failure", line 161 pipeline returns `False`). In the immediate context of line 166, "the results via `delivery_attempts.last_error` (for failed

items)" plainly refers to items that failed the pipeline ? the same items line 161 returns `False` for. There is no reading where "failed items" means transport-failed deliveries; the whole sentence is about recording pipeline "results," and the pipeline produces pass/fail decisions, not transport outcomes.

So the charitable refutation does not hold. The finding stands.

One nuance for severity calibration: this is a design doc (Proposed), and the ADR does offer `policy_decisions` as the alternative store and marks it deferred. But the specific written claim ? that `delivery_attempts.last_error` is a viable store for failed (rejected) items ? is factually wrong given the schema and call graph. It's a false claim in the design contract that would mislead F1/F6/F7 implementation. The finding's downstream point (F7's `/why` has nothing to read until F6; persisted explainability for rejections has no F1?F5 store) is also correct, though I'd note the ADR *does* already list F7?F6 in the phasing table, so that dependency isn't broken ? only the F1?F5 "no persistence for rejections" gap and the false `last_error` claim are the real defects. That keeps it high (false claim in a design contract) rather than merely medium.

14. user (2026-07-06T19:19:28.833Z)

Structured output provided successfully